

EX NO: 1	Python Program to Print Prime Numbers from m to n
DATE:	

Aim:

To write a Python program to print prime numbers between **m** and **n** ($m < n$).

Algorithm:

Step 1: Start the program.

Step 2: Declare the variables **m, n, i, j**.

Step 3: Read the values of **m** (starting number) and **n** (ending number) from the user.

Step 4: Check whether **m** is less than **n**. If not, display a message and stop the program.

Step 5: Use a **for loop** to iterate from **m** to **n**.

Step 6: For each number in the range, check whether the number is **greater than 1**.

Step 7: If the number is greater than 1, initialize a loop to check divisibility.

Step 8: Divide the number by all integers from **2** to **number - 1**.

Step 9: If the number is divisible by any value (remainder = 0), it is **not a prime number**.

Step 10: If the number is not divisible by any other number, it is a **prime number**.

Step 11: Display the **prime number** on the screen.

Step 12: Repeat the process for all numbers between **m** and **n**.

Step 13: Stop the program.

Program:

```
m = int(input("Enter starting number: "))
```

```
n = int(input("Enter ending number: "))
```

```
for num in range(m, n+1):
```

```
    if num > 1:
```

```
        for i in range(2, num):
```

```
            if num % i == 0:
```

```
        break
    else:
        print(num)
```

Output:

```
Enter starting number: 10
Enter ending number: 30
11
13
17
19
23
29
```

Result:

The program has been successfully executed_

EX NO:2	Python Program to Create Book Class
DATE:	

Aim:

To create a Book class with properties id, author, and price, create four book objects, and display the details of the books.

Algorithm:

- Step 1: Start the program.
- Step 2: Define a class named Book.
- Step 3: Create a constructor method (init) inside the class to initialize the attributes id, author, and price.
- Step 4: Assign the values of id, author, and price to the object variables using self keyword.
- Step 5: Define a function named display() to print the details of the book.
- Step 6: Create the first object (b1) of the Book class and assign values for id, author, and price.
- Step 7: Create the second object (b2) of the Book class and assign values.
- Step 8: Create the third object (b3) of the Book class and assign values.
- Step 9: Create the fourth object (b4) of the Book class and assign values.
- Step 10: Call the display() function for each object to print the book details.
- Step 11: Display the book information on the console.
- Step 12: Stop the program.

Program:

```

class Book:
    def __init__(self, id, author, price):
        self.id = id
        self.author = author
        self.price = price

    def display(self):
        print("Book ID:", self.id)
        print("Author:", self.author)
        print("Price:", self.price)

b1 = Book(1, "James", 300)
b2 = Book(2, "John", 450)
b3 = Book(3, "Robert", 250)
b4 = Book(4, "David", 500)

```

```
b1.display()  
b2.display()  
b3.display()  
b4.display()
```

Output:

Book ID: 1
Author: James
Price: 300
Book ID: 2
Author: John
Price: 450
Book ID: 3
Author: Robert
Price: 250
Book ID: 4
Author: David
Price: 500

Result:

The Python program to create a Book class and display the details of four books was successfully executed.

EX NO:3	Python Program to List Files and Their Sizes
DATE:	

Aim:

To write a Python program to list the files and their sizes from a specified directory.

Algorithm:

- Step 1: Start the program.
- Step 2: Import the os module which is used for interacting with the operating system.
- Step 3: Read the directory path from the user using the input function.
- Step 4: Store the given path in a variable named path.
- Step 5: Use the os.listdir() function to get the list of files present in the given directory.
- Step 6: Store the list of files in a variable called files.
- Step 7: Use a for loop to iterate through each file in the directory.
- Step 8: Use os.path.join() to combine the directory path and file name.
- Step 9: Use os.path.getsize() to get the size of each file.
- Step 10: Display the file name and its size in bytes.
- Step 11: Repeat the process for all files in the directory.
- Step 12: Stop the program.

Program:

```
import os
path = input("Enter directory path: ")
files = os.listdir(path)
for f in files:
    size = os.path.getsize(os.path.join(path, f))
    print(f, "-", size, "bytes")
```

Output:

```
Enter directory path: C:\Python
file1.txt - 120 bytes
file2.py - 250 bytes
file3.doc - 500 bytes
```

Result:

The Python program to list files and their sizes from a directory was successfully executed and verified.

EX NO: 4	Python Program to Create a Simple GUI Calculator
DATE:	

Aim:

To develop a **simple calculator using Python GUI (Tkinter)** to perform addition of two numbers.

Algorithm:

- Step 1: Start the program.
- Step 2: Import the tkinter module to create the graphical user interface.
- Step 3: Create the main window using Tk() function.
- Step 4: Declare variables to store the two input numbers and the result.
- Step 5: Create entry widgets to accept two numbers from the user.
- Step 6: Define a function named add() to perform the addition operation.
- Step 7: Inside the function, read the numbers entered by the user.
- Step 8: Add the two numbers and store the result.
- Step 9: Display the result in the result entry box.
- Step 10: Create a button widget to perform the addition operation when clicked.
- Step 11: Run the program using mainloop() to display the GUI window.
- Step 12: Stop the program.

Program:

```
from tkinter import *
def add():
    result.set(int(a.get()) + int(b.get()))
root = Tk()

a = StringVar()
b = StringVar()
result = StringVar()

Entry(root, textvariable=a).pack()
Entry(root, textvariable=b).pack()
Button(root, text="Add", command=add).pack()
Entry(root, textvariable=result).pack()

root.mainloop()
```

Output:

A GUI window will appear.
Enter First Number : 5
Enter Second Number : 3
Result : 8

RCAS

Result:

Thus the above program is executed successfully and the output is obtained.

EX NO: 5	Python Program to Create a GUI Login Authentication
DATE:	

Aim:

To develop a **GUI login screen using Python (Tkinter)** and authenticate the user with **username and password**.

Algorithm:

- Step 1: Start the program.
- Step 2: Import the tkinter module to create the graphical user interface.
- Step 3: Create the main window using Tk() function.
- Step 4: Declare variables to store the username and password.
- Step 5: Create label widgets to display "Username" and "Password".
- Step 6: Create entry widgets to allow the user to enter the username and password.
- Step 7: Define a function named login() to check the entered username and password.
- Step 8: Inside the function, compare the entered username and password with predefined values.
- Step 9: If the username is client and password is server, display Login Successful.
- Step 10: Otherwise, display Login Failed.
- Step 11: Create a Login button to call the login() function.
- Step 12: Use mainloop() to run the GUI window.
- Step 13: Stop the program.

Program:

```
from tkinter import *

def login():
    if user.get()=="client" and pwd.get()=="server":
        print("Login Successful")
    else:
        print("Login Failed")

root = Tk()

user = StringVar()
pwd = StringVar()

Label(root, text="Username").pack()
Entry(root, textvariable=user).pack()
```



```
Label(root, text="Password").pack()  
Entry(root, textvariable=pwd).pack()  
Button(root, text="Login", command=login).pack()  
  
root.mainloop()
```

Output:

Username : client
Password : server
Login Successful

Result:

The Python program to create a GUI login authentication using Tkinter was successfully executed and verified.

EX NO: 6	Python Program to Create "Student" Table in MySQL and Insert Records
DATE:	

Aim:

To write a Python script to create a "Student" table in MySQL, insert at least 3 records, and display them.

Algorithm:

Step 1: Start the program.
Step 2: Import mysql.connector module.
Step 3: Establish connection to MySQL server.
Step 4: Create a cursor object.
Step 5: Create a database (if not exists).
Step 6: Create "Student" table with columns sno, sname, result.
Step 7: Insert minimum 3 records into the table.
Step 8: Execute SELECT query to fetch all records.
Step 9: Display the records on the console. Step 10: Close the connection. Step 11: Stop the program.

Program:

```
import mysql.connector
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your password" # Use your actual MySQL password
)
cursor = conn.cursor()
cursor.execute("CREATE DATABASE IF NOT EXISTS testdb")
cursor.execute("USE testdb")
cursor.execute("CREATE TABLE IF NOT EXISTS Student (sno INT, sname VARCHAR(50),
result VARCHAR(20))")
cursor.execute("INSERT INTO Student VALUES (1, 'Arun', 'Pass')")
cursor.execute("INSERT INTO Student VALUES (2, 'Bala', 'Pass')")
cursor.execute("INSERT INTO Student VALUES (3, 'Charan', 'Fail')")

conn.commit()

cursor.execute("SELECT * FROM Student")
for row in cursor:
    print(row)
conn.close()
```

Output:

```
(1, 'Arun', 'Pass')
(2, 'Bala', 'Pass')
(3, 'Charan', 'Fail')
```

RCAS

Result:

The Student table was created successfully, records were inserted and displayed.

EX NO: 7	Python Program for Simple Chat Application (Socket Programming)
DATE:	

Aim:

To develop a simple chat application using Python networking.

Algorithm:

- Step 1: Start the program.
- Step 2: Import socket module.
- Step 3: Create server socket and bind to port.
- Step 4: Listen for client connection.
- Step 5: Accept connection from client.
- Step 6: Exchange messages between server and client.
- Step 7: Display received messages.
- Step 8: Stop when user exits.
- Step 9: Close connection.
- Step 10: Stop the program.

Program:**#Server Code**

```
import socket
```

```
server = socket.socket()
server.bind(('localhost', 54321))
server.listen(1)
print("Waiting for connection...")
conn, addr = server.accept()
print("Connected to", addr)
```

```
while True:
```

```
    msg = conn.recv(1024).decode()
    print("Client:", msg)
    reply = input("You: ")
    conn.send(reply.encode())
```

#Client Code

```
import socket
```

```
client = socket.socket()
client.connect(('localhost', 54321))
```

```
while True:
```

```
    msg = input("You: ")
    client.send(msg.encode())
```

```
reply = client.recv(1024).decode()  
print("Server:", reply)
```

Output From Client:

You: hello
Server: how are you
You: fine you

Output From Server:

Client: hello
You: how are you
Client: fine you

Result:

Simple chat communication between client and server was successfully implemented.

EX NO: 8	Welcome Application Using Python
DATE:	

Aim:

To develop a Python web application that displays a welcome message with user name.

Algorithm:

- Step 1: Start the application.
- Step 2: Import Flask module.
- Step 3: Create a Flask app.
- Step 4: Create route to accept name input.
- Step 5: Get name using request.
- Step 6: Display welcome message.
- Step 7: Stop the application.

PROGRAM:**Python Code (app.py)**

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/')
def index():
    return """
    <html>
    <body>
        <form action="/greet" method="get">
            Enter Name: <input type="text" name="name">
            <input type="submit" value="Submit">
        </form>
    </body>
    </html>
    """

@app.route('/greet')
def greet():
    name = request.args.get('name', 'Guest')
    return f'<h2>Welcome {name}</h2>'

if __name__ == '__main__':
    print("Flask app starting on http://127.0.0.1:5000")
    app.run(debug=True, host='127.0.0.1', port=5000)
```

Output:

Enter Name: John
Welcome John

RCAS

Result:

The Python web application successfully displays the welcome message using user input.

EX NO: 9	Design and Develop Ruby on Rails Web Application to Manipulate Book Details using MySQL Database
DATE:	

Aim:

To design and develop a Ruby on Rails web application to manage (add and display) book details using MySQL database.

Algorithm:

- Step 1: Start the application.
- Step 2: Create a new Rails application.
- Step 3: Configure MySQL database in database.yml.
- Step 4: Generate Book model with fields (title, author, price).
- Step 5: Run migration to create the Book table in MySQL.
- Step 6: Generate controller for Book.
- Step 7: Create actions for adding and listing books.
- Step 8: Create views for entering and displaying book details.
- Step 9: Insert book data using form.
- Step 10: Retrieve and display all book records.
- Step 11: Stop the application.

PROGRAM:**Create Model**

```
rails generate model Book title:string author:string price:float
rails db:migrate
```

Controller (books_controller.rb)

```
class BooksController < ApplicationController
  def new
    @book = Book.new
  end
  def create
    @book = Book.new(book_params)
    @book.save
    redirect_to books_path
  end
  def index
    @books = Book.all
  end
  private
  def book_params
    params.require(:book).permit(:title, :author, :price)
  end
end
```


end

View: Add Book (new.html.erb)

```
<h2>Add Book Details</h2>
<form action="/books" method="post">
  Title: <input type="text" name="book[title]"><br><br>
  Author: <input type="text" name="book[author]"><br><br>
  Price: <input type="text" name="book[price]"><br><br>
  <input type="submit" value="Add Book">
</form>
```

View: List Books (index.html.erb)

```
<h2>Book Details</h2>
<table border="1">
<tr>
  <th>Title</th>
  <th>Author</th>
  <th>Price</th>
</tr>
<% @books.each do |b| %>
<tr>
  <td><%= b.title %></td>
  <td><%= b.author %></td>
  <td><%= b.price %></td>
</tr>
<% end %>
</table>
```

Output:

```
Add Book Details
Title: Python
Author: John
Price: 500
Book Details
Python John 500
```

Result:

The Ruby on Rails application was successfully developed to add and display book details using

EX NO: 10	Python Server to Accept Two Numbers from Browser and Return Their Sum
DATE:	

Aim:

To create a Python web server that accepts two numbers from a browser and returns their sum.

Algorithm:

- Step 1: Start the program.
- Step 2: Import the Flask module to create a web server.
- Step 3: Create a Flask application object.
- Step 4: Define a route ('/') to display an HTML form in the browser.
- Step 5: Create another route ('/add') to handle form submission.
- Step 6: Retrieve the two numbers entered by the user using request.form.
- Step 7: Convert the input values into integers.
- Step 8: Add the two numbers and store the result.
- Step 9: Return the result as a response to the browser.
- Step 10: Run the Flask server.
- Step 11: Open the browser and enter the server URL.
- Step 12: Stop the program.

Program:

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/')
def home():
    return """
    <h2>Add Two Numbers</h2>
    <form action="/add" method="post">
        Number 1: <input type="text" name="num1"><br><br>
        Number 2: <input type="text" name="num2"><br><br>
        <input type="submit" value="Add">
    </form>
    """

@app.route('/add', methods=['POST'])
def add():
    num1 = int(request.form['num1'])
    num2 = int(request.form['num2'])
    result = num1 + num2
    return f"<h3>Result: {result}</h3>"

if __name__ == '__main__':
```

```
print("Flask app starting on http://127.0.0.1:5000")
app.run(debug=True, host='127.0.0.1', port=5000)
```

Output:

Add Two Numbers

Number 1:

Number 2:

Result: 50

Result:

The Python server was successfully created using Flask. It accepts two numbers from the browser and correctly returns their sum.